

# Deep Learning for Solving Dynamic Models

## Session 8 — Economics-informed neural networks: foundations

Simon Scheidegger

HEC, University of Lausanne

Based on Raissi, Perdikaris & Karniadakis (2019), *Journal of Computational Physics* 378, 686–707

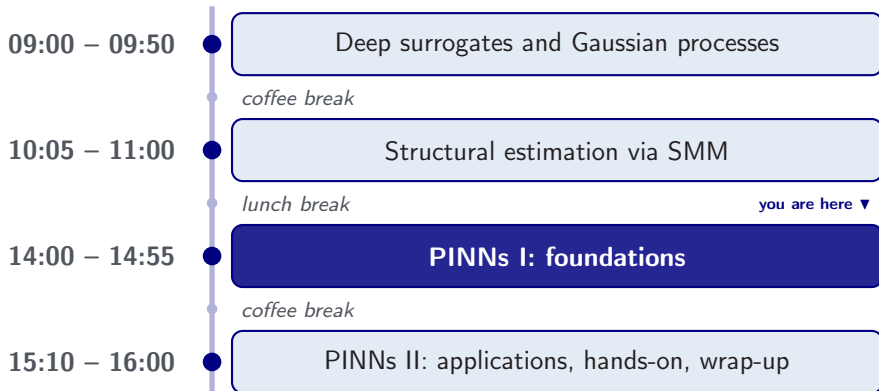
University of Padova

September 24, 2026

Further: Sirignano & Spiliopoulos (2018), Deep Galerkin Method for PDEs in high dimensions.

Al-Aradi, Correia, Naez, Hoshisashi & Schwab (2018), DGM PDE solver.

# Day 2 at a Glance



# Outline

## 1. From DEQNs to PINNs

Bridging discrete and continuous time; the PINN loss.

## 2. PINN Fundamentals

Automatic differentiation for PDEs, collocation, training loop.

## 3. Soft vs. Hard Boundary Conditions

Trial solutions, 2D Poisson, practical guidance.

## 4. Economic Applications: HJB Equation

Cake-eating problem, scaled trial solution + hard BCs, Adam  $\rightarrow$  L-BFGS, analytical benchmark; DGM as optional high-D architecture.

## 5. Financial Applications: Black–Scholes

Option pricing via PINNs, comparison with analytical formula.

## 6. When to Use One, and What Goes Wrong

PINNs vs. grid solvers, inverse problems, failure modes, and which notebooks to run.

# Roadmap of This Session (55 min)

**Part I of two.** This session builds the method; Session 9 after the break applies it to economic and financial models.

## I. From DEQNs to PINNs (12 min)

- ▶ Discrete vs. continuous time, the PINN loss
- ▶ Finger exercise

## II. PINN fundamentals (20 min)

- ▶ Autograd for PDEs, collocation, the training loop
- ▶ Adam  $\rightarrow$  L-BFGS in FP64; warm-up 1D ODE

## III. Boundary conditions (23 min)

- ▶ Soft penalties vs. hard trial solutions
- ▶ Hard BCs in 2D; the Poisson equation; finger exercise

# Part I

From DEQNs to PINNs

# Recall: The DEQN Loss from Day 1

**Yesterday morning** (02\_DEQN.pdf) **we trained** a neural network  $\mathcal{N}_\theta$  to satisfy economic equilibrium conditions in **discrete time**:

$$\ell_\theta^{\text{DEQN}} = \frac{1}{N} \sum_{i=1}^N \|G(\mathbf{x}_i, \mathcal{N}_\theta(\mathbf{x}_i))\|^2$$

- ▶  $G(\cdot) = 0$ : Euler equations, budget constraints, market clearing
- ▶ Training points  $\mathbf{x}_i$ : simulated from the model's own dynamics
- ▶ **No labeled data needed**, structural equations *are* the loss

**Today's question:** What if the economic model is formulated in **continuous time**?

# Discrete vs. Continuous Time

## Discrete Time (DEQN)

Euler equation:

$$u'(c_t) = \beta \mathbb{E}[u'(c_{t+1}) R_{t+1}]$$

State transition:

$$\mathbf{x}_{t+1} = h(\mathbf{x}_t, p(\mathbf{x}_t), \varepsilon_{t+1})$$

Loss: algebraic residual  $G(\cdot) = 0$

## Continuous Time (PINN)

State evolution:

$$da = (ra - c) dt + \sigma dW$$

HJB equation:

$$\rho V = \max_c \{ u(c) + V_a(ra - c) + \frac{1}{2}\sigma^2 V_{aa} \}$$

Loss: PDE residual  $\mathcal{R}(\cdot) = 0$

**Key shift:** from algebraic residuals to **differential** residuals, we need **derivatives** of the neural network. The diffusion term  $\frac{1}{2}\sigma^2 V_{aa}$  is why **second** derivatives matter.

# Why Continuous Time Matters

Many frontier models are formulated in continuous time:

- ▶ **Heterogeneous-agent models:** Achdou, Han, Lasry, Lions & Moll (2022), income and wealth distribution dynamics via HJB–KFE systems
- ▶ **Mean-field games:** optimal policy with a continuum of interacting agents
- ▶ **Asset pricing:** Black–Scholes, Heston, jump-diffusion models
- ▶ **Macro-finance:** Brunnermeier & Sannikov (2014), He & Krishnamurthy (2013)

## Where this sits in the summer school

Day 1 solved equilibrium conditions in **discrete** time with DEQNs. This morning we do the **continuous-time** counterpart, and at 11:00 Yang takes the same machinery to heterogeneous-agent models, where the HJB is coupled to a Kolmogorov forward equation.

# The PINN Loss Function

**Core idea** (Raissi, Perdikaris & Karniadakis, 2019):

Given a PDE  $\mathcal{D}[u](\mathbf{x}) = 0$  on domain  $\Omega$  with boundary conditions  $\mathcal{B}[u](\mathbf{x}) = 0$  on  $\partial\Omega$ , define:

$$\ell_{\theta}^{\text{PINN}} = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{D}[\mathcal{N}_{\theta}](\mathbf{x}_i^r)|^2}_{\text{PDE residual}} + \underbrace{\frac{\lambda}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}[\mathcal{N}_{\theta}](\mathbf{x}_j^b)|^2}_{\text{boundary/initial conditions}}$$

- ▶  $\mathcal{N}_{\theta}$ : neural network with parameters  $\theta$
- ▶  $\mathbf{x}_i^r$ : **collocation points** in the interior of  $\Omega$
- ▶  $\mathbf{x}_j^b$ : points on the boundary  $\partial\Omega$
- ▶ Derivatives  $\mathcal{D}[\mathcal{N}_{\theta}]$  computed via **automatic differentiation**

**Warm-up ODE:** for  $u''(x) = -1$  on  $(0, 1)$  with  $u(0) = u(1) = 0$ , use  $\mathcal{D}[u] = u'' + 1$  in the interior and  $\mathcal{B}[u] = (u(0), u(1))$  on the boundary.

# Finger Exercise: Writing a PINN Loss

## Exercise (5 min)

Consider the ODE  $u''(x) = -1$  on  $[0, 1]$  with boundary conditions  $u(0) = u(1) = 0$ .

1. Write down the PINN residual  $\mathcal{R}(x; \theta) = \dots$  using a neural network  $\mathcal{N}(x; \theta)$ .
2. Write the total PINN loss as a sum of a PDE residual term and a boundary term.
3. What is the exact solution?

# Solution: Writing a PINN Loss

## Answer

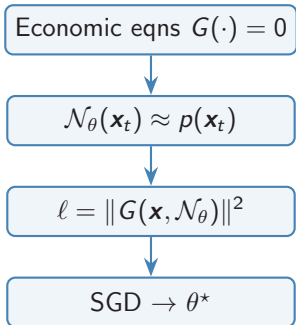
1.  $\mathcal{R}(x; \theta) = \frac{\partial^2 \mathcal{N}}{\partial x^2}(x; \theta) + 1.$

2.  $\mathcal{L}(\theta) = \underbrace{\frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{R}(x_i; \theta)|^2}_{\text{PDE residual}} + \lambda \underbrace{[\mathcal{N}(0; \theta)^2 + \mathcal{N}(1; \theta)^2]}_{\text{boundary conditions}}$

3.  $u(x) = \frac{1}{2}x(1-x)$  satisfies  $u'' = -1$ ,  $u(0) = u(1) = 0$ . ✓

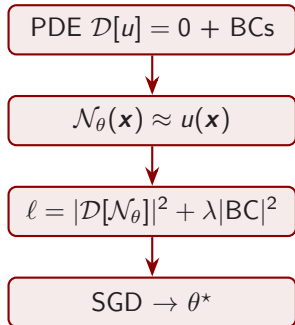
# DEQN vs. PINN: Side-by-Side

## DEQN (Discrete Time)



← same principle →  
+ autograd derivatives

## PINN (Continuous Time)



Both approaches are **unsupervised**: the physics (economics) is the loss function. The key addition for PINNs: we differentiate the network via any auto-diff tool (e.g. `torch.autograd.grad`).

# Part II

## PINN Fundamentals

# Automatic Differentiation for PDEs

**Key tool:** `torch.autograd.grad` with `create_graph=True`.

```
# First derivative dy/dx
dydx = torch.autograd.grad(
    y, x,
    grad_outputs=torch.ones_like(y),
    create_graph=True # keep graph for higher derivs
)[0]
```

- ▶ `create_graph=True`: retains the computational graph so we can differentiate **again**
- ▶ This gives us **algorithmic** derivatives (up to floating-point precision), no finite differences!
- ▶ Works for **any order**:  $\frac{\partial^n}{\partial x^n}$  by chaining  $n$  calls

# Computing Second Derivatives

Chain two autograd calls to get  $y''(x)$ :

*# First derivative*

```
dydx = torch.autograd.grad(  
    y, x, grad_outputs=torch.ones_like(y),  
    create_graph=True)[0]
```

*# Second derivative*

```
d2ydx2 = torch.autograd.grad(  
    dydx, x, grad_outputs=torch.ones_like(dydx),  
    create_graph=True)[0]
```



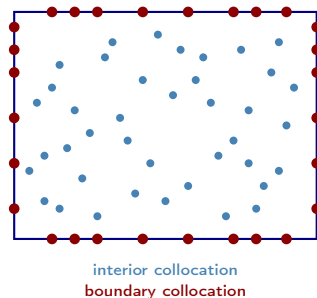
# Collocation Points

PINNs evaluate the PDE residual at **randomly sampled** points, no grid required.

1D: interior + boundary



2D:  $\Omega = (0, 1)^2$



Points are **resampled** periodically during training to avoid overfitting to a fixed set.

# PINN Training Algorithm

## Algorithm: Physics-Informed Neural Network Training

**Input:** PDE  $\mathcal{D}[u] = 0$ , BCs  $\mathcal{B}[u] = 0$ , network  $\mathcal{N}_\theta$ , learning rate  $\eta$

**for** epoch =  $1, \dots, E$  **do**

    Sample  $N_r$  collocation points  $\{\mathbf{x}_i^r\}$  in domain  $\Omega$

    Sample  $N_b$  boundary points  $\{\mathbf{x}_j^b\}$  on  $\partial\Omega$

    Forward pass: compute  $\mathcal{N}_\theta(\mathbf{x}_i^r)$  and  $\mathcal{N}_\theta(\mathbf{x}_j^b)$

    Autograd: compute  $\mathcal{D}[\mathcal{N}_\theta](\mathbf{x}_i^r)$  via `torch.autograd.grad`

$\ell_\theta = \frac{1}{N_r} \sum_i |\mathcal{D}[\mathcal{N}_\theta](\mathbf{x}_i^r)|^2 + \frac{\lambda}{N_b} \sum_j |\mathcal{B}[\mathcal{N}_\theta](\mathbf{x}_j^b)|^2$

    Update:  $\theta \leftarrow \theta - \eta \nabla_\theta \ell_\theta$

**end for**

**Output:** Trained  $\mathcal{N}_{\theta^*}$  approximating the PDE solution

**Note:** Unlike DEQNs, training points are **not** simulated from model dynamics, they are drawn uniformly (or adaptively) from the domain.

# Optimizer Pipeline: Adam $\rightarrow$ L-BFGS, in FP64

## Two stages.

- ▶ **Adam** on resampled collocation: finds a basin.
- ▶ **Deterministic L-BFGS** on a *fixed* grid: closes the residual. Resampling per evaluation breaks the line search.

## Why FP64?

- ▶ tanh-MLP second derivatives lose  $\sim 7$  digits in FP32.
- ▶ The Wolfe search compares values at exactly that scale.
- ▶ FP32 plateaus at the noise floor; FP64 keeps going.

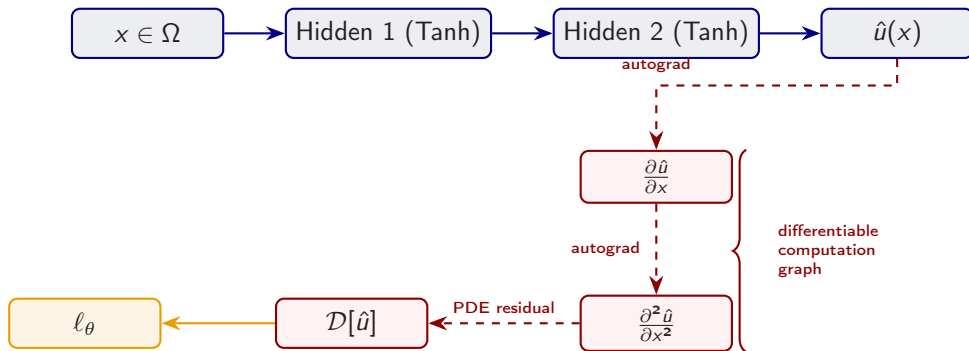
Used in notebooks 04 and 05. Report PDE residuals and solution diagnostics, not the optimizer label.

## Practical recipe

1. Cast model + data to float64.
2. Adam, lr  $10^{-3}$ , a few k epochs, resampled collocation.
3. Freeze grid; `torch.optim.LBFGS` (strong-Wolfe).
4. Stop on residual stagnation.

Precision evidence: [arXiv:2505.10949](#).

# PINN Architecture



The entire pipeline, forward pass *and* derivative computation, is **differentiable**, enabling end-to-end gradient-based training.

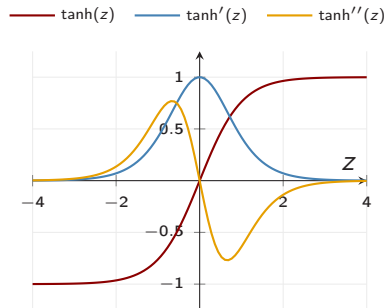
# Why Tanh Activations for PINNs?

PDE residuals require smooth derivatives.

- ▶ **ReLU**:  $C^0$  only; its second derivative vanishes almost everywhere, so a  $u''$  residual carries no gradient
- ▶ **Tanh**:  $C^\infty$ , all derivatives are well-defined and informative
- ▶ **Softplus**:  $C^\infty$ , useful for positivity constraints (e.g. consumption)

## Rule of thumb

For PINNs requiring  $k$ -th order derivatives, use activations that are at least  $C^k$ .



# Warm-up: Solving an ODE with a PINN

**Problem:**  $y''(x) = -1$  on  $(0, 1)$ ,  $y(0) = y(1) = 0$ .    **Analytical:**  $y(x) = \frac{1}{2}x(1 - x)$ .

## PINN setup:

1. NN  $\mathcal{N}_\theta : [0, 1] \rightarrow \mathbb{R}$ , 2 hidden layers of 20 units, tanh
2. Residual:  $r(x) = \mathcal{N}_\theta''(x) + 1$
3. Loss: PDE residual + BC penalty (soft, here)

$$\ell = \frac{1}{N} \sum_i r(x_i)^2 + \mathcal{N}_\theta(0)^2 + \mathcal{N}_\theta(1)^2$$

## After 5,000 Adam epochs:

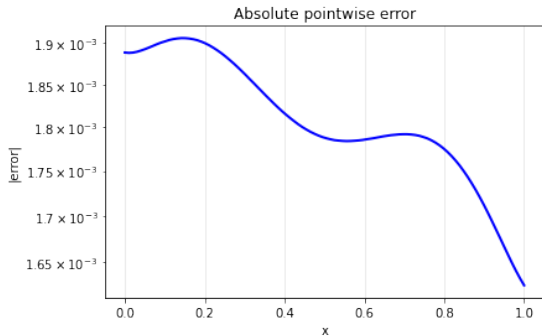
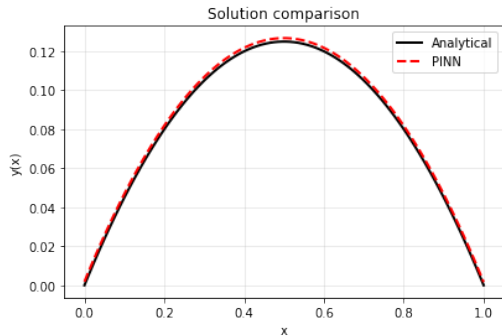
- ▶ Final loss  $\approx 2 \times 10^{-5}$ , of which the BC term is  $\approx 9 \times 10^{-6}$
- ▶ Max absolute error  $1.9 \times 10^{-3}$ , mean  $1.8 \times 10^{-3}$

## Read the error, not the loss

A small loss says the residual is small at the sampled points. Only the comparison against  $\frac{1}{2}x(1 - x)$  says the *solution* is right.

**Notebook:** 01\_ODE\_PINN\_ZeroBCs

# Warm-up: PINN vs. Analytical Solution



Left: the network on top of  $y(x) = \frac{1}{2}x(1-x)$  — indistinguishable at this scale. Right: the absolute error, which is what actually tells you how well it did. Note the axis: it stays in a narrow band around  $1.8 \times 10^{-3}$  across the domain, with no drift to the boundaries.

**Notebook:** 01\_ODE\_PINN\_ZeroBCs

# Non-Zero Boundary Conditions: The Challenge

**Problem:**  $y''(x) = -1$  on  $(0, 1)$  with  $y(0) = 1$ ,  $y(1) = 2$ .

**Analytical solution:**  $y(x) = -\frac{1}{2}x^2 + \frac{3}{2}x + 1$ .

**Soft BC enforcement:**

$$\ell = \frac{1}{N} \sum_i r(x_i)^2 + \lambda_0 (\mathcal{N}_\theta(0) - 1)^2 + \lambda_1 (\mathcal{N}_\theta(1) - 2)^2$$

**Issues with soft enforcement:**

- ▶ BCs are only **approximately** satisfied
- ▶ Penalty weight  $\lambda$  requires **tuning**: too small  $\Rightarrow$  BC violated; too large  $\Rightarrow$  PDE ignored
- ▶ Boundary errors can **propagate** into the interior

$\Rightarrow$  **Motivates hard boundary enforcement** (Part III).

# Part III

Soft vs. Hard Boundary Conditions

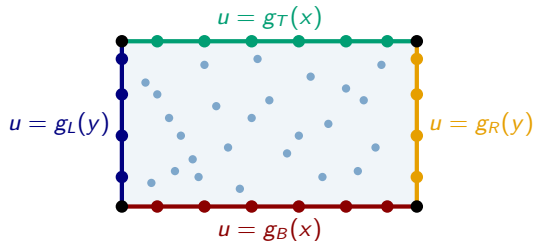
# What Are Boundary Conditions?

**1D:**  $y''(x) = -1$  on  $(0, 1)$



Interior: enforce  $y'' + 1 = 0$   
Boundary: enforce  $y = \text{prescribed value}$

**2D:**  $\Delta u = f$  on  $(0, 1)^2$

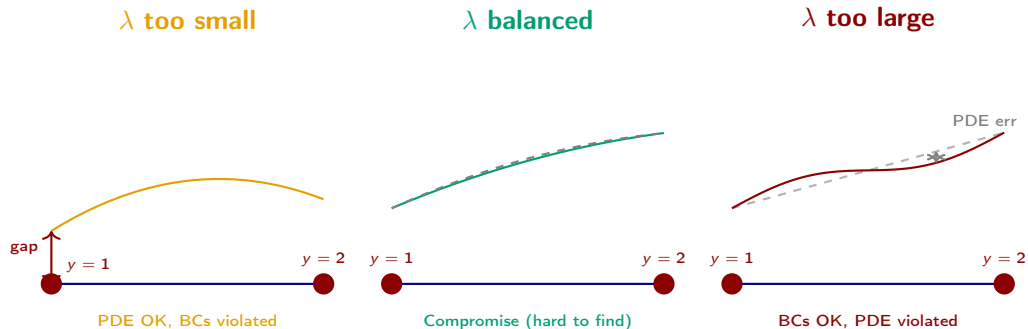


## Two enforcement strategies

**Soft:** add BC violation as a penalty term in the loss.

**Hard:** build BCs into the network output by construction.

# The Problem with Soft Boundary Conditions



Soft BCs require **careful balancing** of the penalty weight  $\lambda$ .  
Too small: BCs violated. Too large: PDE ignored. Just right: hard to find *a priori*.

# Hard BC Enforcement: Trial Solutions

**Idea:** construct a **trial solution** that satisfies BCs *by construction*:

$$\hat{y}(x) = A(x) + B(x) \cdot \mathcal{N}_\theta(x)$$

where:

- ▶  $A(x)$ : **anchor function** satisfying the BCs exactly, e.g.  $A(x) = 1 + x$
- ▶  $B(x)$ : **mask function** vanishing at boundaries, e.g.  $B(x) = x(1 - x)$
- ▶  $\mathcal{N}_\theta(x)$ : free neural network (no BC constraints)

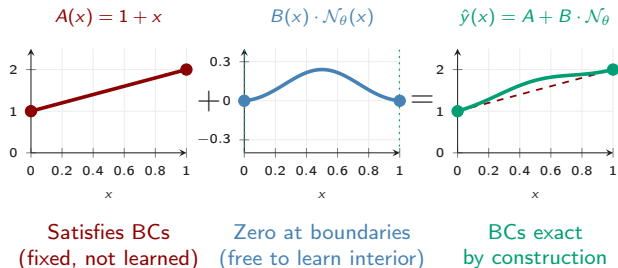
**Verification:**

$$\hat{y}(0) = A(0) + B(0) \cdot \mathcal{N}_\theta(0) = 1 + 0 = 1 \quad \checkmark$$

$$\hat{y}(1) = A(1) + B(1) \cdot \mathcal{N}_\theta(1) = 2 + 0 = 2 \quad \checkmark$$

$\Rightarrow$  BCs hold **identically**, only the PDE residual loss remains.

# Trial Solution: Visual Decomposition



# Hard BC: Implementation

```
def A(x): # Anchor: A(0)=1, A(1)=2
    return 1.0 + x
def B(x): # Mask: B(0)=B(1)=0
    return x * (1.0 - x)

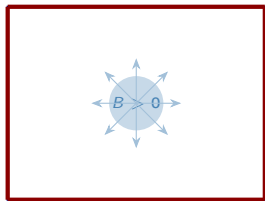
def y_trial(model, x): # Hard-BC trial solution
    return A(x) + B(x) * model(x)

def residual(model, x):
    x.requires_grad_(True)
    y = y_trial(model, x)
    dydx = torch.autograd.grad(y, x, ...)[0]
    d2ydx2 = torch.autograd.grad(dydx, x, ...)[0]
    return d2ydx2 + 1.0 # should be 0
```

**Advantage:** the loss has **only one term** (PDE residual), no  $\lambda$  to tune!

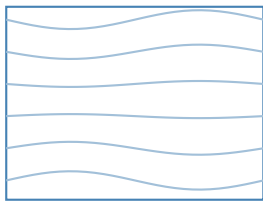
# Hard BCs in 2D: Building the Trial Solution

**Mask**  $B(x, y) = x(1-x)y(1-y)$



$B = 0$  on all edges

**Network**  $\mathcal{N}_\theta(x, y)$



Free to take any value



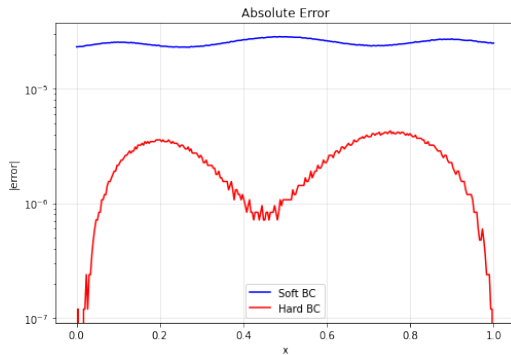
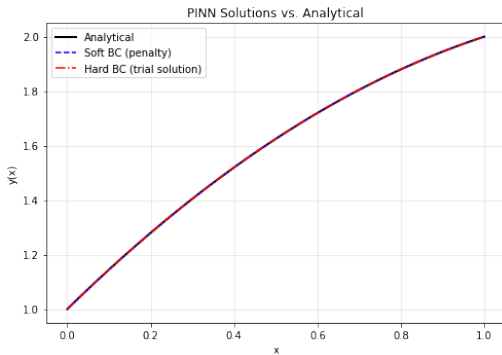
$B \cdot \mathcal{N}_\theta$ : zero on  $\partial\Omega$



Exactly zero on boundary

Then  $\hat{u}(x, y) = \underbrace{A(x, y)}_{\text{matches BCs}} + \underbrace{B(x, y) \cdot \mathcal{N}_\theta(x, y)}_{\text{zero on } \partial\Omega}$  satisfies Dirichlet BCs **exactly**.

# Soft vs. Hard BCs: Comparison



Both fit the solution (left). The difference is the error (right, log scale): the soft-BC run sits near  $10^{-5}$  **including at the endpoints**, while the hard-BC run falls to  $10^{-7}$  at the boundary, exact there by construction.

**Notebook:** 02\_ODE\_PINN\_SoftVsHardBCs

# Finger Exercise: Hard Boundary Conditions

## Exercise (5 min)

Consider the same ODE  $u''(x) = -1$  on  $[0, 1]$  with  $u(0) = u(1) = 0$ .

1. Construct a **hard-BC trial solution** of the form  $\hat{u}(x) = B(x) \cdot \mathcal{N}(x; \theta)$ , where  $B(x)$  enforces both BCs exactly. (Hint:  $B$  must vanish at  $x = 0$  and  $x = 1$ .)
2. Verify that  $\hat{u}(0) = \hat{u}(1) = 0$  for *any* network output  $\mathcal{N}$ .
3. What is the advantage of hard BCs over soft BCs?

# Solution: Hard Boundary Conditions

## Answer

1.  $\hat{u}(x) = x(1 - x) \cdot \mathcal{N}(x; \theta)$ . Since  $B(x) = x(1 - x)$  vanishes at both endpoints, the BCs are satisfied exactly.
2.  $\hat{u}(0) = 0 \cdot \mathcal{N}(0; \theta) = 0$ .  $\hat{u}(1) = 0 \cdot \mathcal{N}(1; \theta) = 0$ . ✓  
This holds regardless of what  $\mathcal{N}$  outputs.
3. Hard BCs **eliminate the boundary loss term** entirely, the optimizer only needs to minimize the PDE residual. This reduces the number of hyperparameters ( $\lambda$  is gone) and often speeds up convergence.

# Extension to 2D: Poisson Equation

**Problem:**  $\Delta u = f(x, y)$  on  $\Omega = (0, 1)^2$  with Dirichlet BCs.

**Manufactured solution:**  $u^*(x, y) = x^2 + y + \sin(\pi x) \sin(\pi y)$ .

**Hard BCs via transfinite (Coons-patch) interpolation**, the standard 4-side construction that matches each boundary function exactly along its edge:

$$\hat{u}(x, y) = A(x, y) + \underbrace{x(1-x)y(1-y)}_{B(x,y)} \cdot \mathcal{N}_\theta(x, y)$$

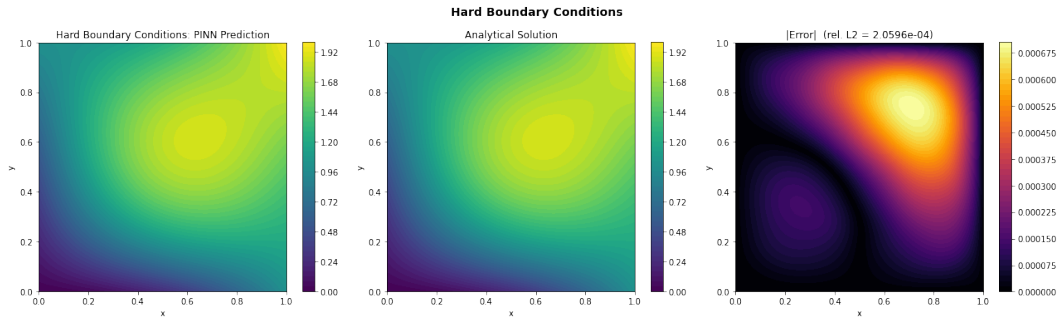
where  $A(x, y)$  interpolates boundary data from all four sides:

$$A = (1-x)g_L(y) + xg_R(y) + (1-y)g_B(x) + yg_T(x) - \text{corners}$$

**Laplacian via autograd:**

$$\Delta \hat{u} = \frac{\partial^2 \hat{u}}{\partial x^2} + \frac{\partial^2 \hat{u}}{\partial y^2} \quad (\text{two chains of autograd.grad})$$

# 2D Poisson: Results



Hard-BC PINN, analytical solution, and absolute error. Relative  $L^2$  error  $2.06 \times 10^{-4}$  against  $1.36 \times 10^{-3}$  for the tuned soft-penalty run: a **6.6×** improvement, at one fewer hyperparameter.

**Notebook:** 03\_PDE\_PINN\_Poisson2D

# Soft vs. Hard BCs: When to Use Which?

|                 | Soft BCs                                | Hard BCs                                                  |
|-----------------|-----------------------------------------|-----------------------------------------------------------|
| <b>Best for</b> | Irregular domains, Robin or Neumann BCs | Simple domains, Dirichlet BCs, high accuracy              |
| <b>Pros</b>     | Flexible, easy to implement             | Exact BCs, fewer hyperparams, better accuracy             |
| <b>Cons</b>     | $\lambda$ tuning, approximate BCs       | Needs analytical $A$ , $B$ ; harder for irregular domains |

## Practical recommendation

Start with **hard BCs** when the domain and boundary data allow it. Fall back to **soft BCs** for complex geometries or mixed boundary types.